

LABSHEET - 6**NITIN PANWAR****BTECH(CSE) 3RD YEAR****CU240250967****SECTION - B****ROLL NO. - 39****Mastery in Data Analytics and Visualizations and Career Advancement (Technical)**

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
import sys
```

```
# EXPERIMENT 6
# IMAGE SEGMENTATION TECHNIQUES
# =====

print("=" * 65)
print("EXPERIMENT 6")
print("Implementation and Comparative Analysis of")
print("Image Segmentation Techniques")
print("=" * 65)
print("NITIN PANWAR")
print("CU240250967")
```

```
=====
EXPERIMENT 6
Implementation and Comparative Analysis of
Image Segmentation Techniques
=====
HIMANI CHAUDHARY
CU240250967
```

```
# 1. LOAD IMAGE
# =====

image_path = "/content/beautiful-nature-mountain-scenery-with-flowers-picjumbo-com.jpg"

image = cv2.imread(image_path)

if image is None:
    print("\nERROR: Image not found!")
    print("Please check your image path:")
    print(image_path)
    sys.exit()

print("\nImage loaded successfully!")
print("Image size:", image.shape)

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
print("HIMANI CHAUDHARY")
print("CU240250967")
```

```
Image loaded successfully!
Image size: (3584, 5376, 3)
HIMANI CHAUDHARY
CU240250967
```

```
# 2. GRAYSCALE CONVERSION
# =====

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

print("Grayscale conversion completed.")
print("HIMANI CHAUDHARY")
print("CU240250967")
```

Grayscale conversion completed.
HIMANI CHAUDHARY
CU240250967

```
# 3. GAUSSIAN BLUR
# =====

blur = cv2.GaussianBlur(gray, (5, 5), 0)

print("Gaussian Blur preprocessing completed.")
print("HIMANI CHAUDHARY")
print("CU240250967")
```

Gaussian Blur preprocessing completed.
HIMANI CHAUDHARY
CU240250967

```
# 4. DISPLAY IMAGES
# =====

plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(blur, cmap="gray")
plt.title("Gaussian Blur")
plt.axis("off")

plt.tight_layout()
plt.show()
print("HIMANI CHAUDHARY")
print("CU240250967")
```

Original Image



Grayscale Image



Gaussian Blur



HIMANI CHAUDHARY
CU240250967

```
# 5. GLOBAL THRESHOLDING
# =====

print("\n" + "=" * 65)
print("GLOBAL THRESHOLDING")
print("=" * 65)

threshold_value = 127

_, global_threshold = cv2.threshold(
    blur,
    threshold_value,
    255,
    cv2.THRESH_BINARY
)

print("Global threshold value:", threshold_value)
print("HIMANI CHAUDHARY")
```

```
print("CU240250967")
```

```
=====
GLOBAL THRESHOLDING
=====
```

```
Global threshold value: 127
HIMANI CHAUDHARY
CU240250967
```

```
# 6. OTSU'S THRESHOLDING
```

```
# =====
```

```
print("\n" + "=" * 65)
print("OTSU'S THRESHOLDING")
print("=" * 65)
```

```
otsu_value, otsu_threshold = cv2.threshold(
    blur,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

```
print("Automatically selected Otsu threshold:", otsu_value)
print("HIMANI CHAUDHARY")
print("CU240250967")
```

```
=====
OTSU'S THRESHOLDING
=====
```

```
Automatically selected Otsu threshold: 107.0
HIMANI CHAUDHARY
CU240250967
```

```
# 7. ADAPTIVE THRESHOLDING
```

```
# =====
```

```
print("\n" + "=" * 65)
print("ADAPTIVE THRESHOLDING")
print("=" * 65)
```

```
adaptive_threshold = cv2.adaptiveThreshold(
    blur,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)
```

```
print("Adaptive thresholding completed.")
print("HIMANI CHAUDHARY")
print("CU240250967")
```

```
=====
ADAPTIVE THRESHOLDING
=====
```

```
Adaptive thresholding completed.
HIMANI CHAUDHARY
CU240250967
```

```
# 8. DISPLAY THRESHOLDING RESULTS
```

```
# =====
```

```
plt.figure(figsize=(15, 5))
```

```
plt.subplot(1, 3, 1)
plt.imshow(global_threshold, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")
```

```
plt.subplot(1, 3, 2)
plt.imshow(otsu_threshold, cmap="gray")
plt.title("Otsu Thresholding")
plt.axis("off")
```

```
plt.subplot(1, 3, 3)
plt.imshow(adaptive_threshold, cmap="gray")
plt.title("Adaptive Thresholding")
```

```
plt.axis("off")

plt.tight_layout()
plt.show()
print("HIMANI CHAUDHARY")
print("CU240250967")
```

Global Thresholding



Otsu Thresholding



Adaptive Thresholding



HIMANI CHAUDHARY
CU240250967

```
# 9. WATERSHED SEGMENTATION
# =====

print("\n" + "=" * 65)
print("WATERSHED SEGMENTATION")
print("=" * 65)

print("HIMANI CHAUDHARY CU240250967")

binary = otsu_threshold.copy()

kernel = np.ones((3, 3), np.uint8)

# Remove noise
opening = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)

# Sure background
sure_background = cv2.dilate(
    opening,
    kernel,
    iterations=3
)

# Distance transform
distance = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)

# Sure foreground
_, sure_foreground = cv2.threshold(
    distance,
    0.5 * distance.max(),
    255,
    cv2.THRESH_BINARY
)

sure_foreground = np.uint8(sure_foreground)

# Unknown region
unknown = cv2.subtract(
    sure_background,
    sure_foreground
)

# Marker labelling
num_labels, markers = cv2.connectedComponents(
    sure_foreground
```

```

)

markers = markers + 1

markers[unknown == 255] = 0

# Apply watershed
watershed_image = image.copy()

markers = cv2.watershed(
    watershed_image,
    markers
)

# Mark boundaries in red
watershed_result = image_rgb.copy()
watershed_result[markers == -1] = [255, 0, 0]

print("Watershed segmentation completed.")
print("Number of regions:", num_labels - 1)
print("HIMANI CHAUDHARY")
print("CU240250967")

```

```

=====
WATERSHED SEGMENTATION
=====
HIMANI CHAUDHARY CU240250967
Watershed segmentation completed.
Number of regions: 5
HIMANI CHAUDHARY
CU240250967

```

```

# 10. K-MEANS SEGMENTATION
# =====

print("\n" + "=" * 65)
print("K-MEANS IMAGE SEGMENTATION")
print("=" * 65)

print("HIMANI CHAUDHARY    CU240250967")

# Resize image
small_image = cv2.resize(
    image_rgb,
    (256, 256)
)

# Convert pixels into a list
pixels = small_image.reshape((-1, 3))
pixels = np.float32(pixels)

# Number of clusters
k = 4

print("Number of clusters:", k)

# K-Means model
kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

kmeans.fit(pixels)

# Cluster centers
centers = np.uint8(kmeans.cluster_centers_)

# Labels
labels = kmeans.labels_

# Replace pixels with cluster centers
segmented_pixels = centers[labels]

# Reshape
kmeans_result = segmented_pixels.reshape(
    small_image.shape
)

print("K-Means segmentation completed.")

```

```
print("HIMANI CHAUDHARY")
print("CU240250967")
```

```
=====
K-MEANS IMAGE SEGMENTATION
=====
HIMANI CHAUDHARY    CU240250967
Number of clusters: 4
K-Means segmentation completed.
HIMANI CHAUDHARY
CU240250967
```

```
# 11. FINAL COMPARISON
# =====

plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_threshold, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(otsu_threshold, cmap="gray")
plt.title("Otsu Thresholding")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(adaptive_threshold, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(watershed_result)
plt.title("Watershed")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(kmeans_result)
plt.title("K-Means")
plt.axis("off")

plt.tight_layout()
plt.show()
print("HIMANI CHAUDHARY")
print("CU240250967")
```

Original Image



Global Thresholding



Otsu Thresholding



Adaptive Thresholding



Watershed



K-Means



HIMANI CHAUDHARY
CU240250967

```
# 12. OBSERVATION
# =====

print("\n" + "=" * 65)
print("OBSERVATION")
print("=" * 65)

print("""
1. Global Thresholding:
    Separates foreground and background using a fixed threshold.

2. Otsu's Thresholding:
    Automatically selects an optimal threshold value.

3. Adaptive Thresholding:
    Uses local regions and works better when illumination
    is uneven.

4. Watershed:
    Useful for separating touching or overlapping objects.

5. K-Means:
    Groups pixels into different clusters based on their
    color characteristics.

Overall:
Different segmentation techniques are suitable for different
types of images. Thresholding works well for simple images,
Watershed is useful for separating connected objects, and
K-Means is useful for color-based segmentation.
""")

print("\n" + "=" * 65)
print("EXPERIMENT 6 COMPLETED SUCCESSFULLY")
print("=" * 65)
print("HIMANI CHAUDHARY")
print("CU240250967")
```

```
=====
OBSERVATION
=====
```

```
1. Global Thresholding:
    Separates foreground and background using a fixed threshold.
```

2. Otsu's Thresholding:
Automatically selects an optimal threshold value.
3. Adaptive Thresholding:
Uses local regions and works better when illumination is uneven.
4. Watershed:
Useful for separating touching or overlapping objects.
5. K-Means:
Groups pixels into different clusters based on their color characteristics.